

Remote Diagnostics Graph

Interview Mastery Guide

Explain everything end-to-end — product, theory, code, architecture, tools

Beginner-friendly language · Senior depth · Academic theory · Dev & Architect Q&A

Enterprise warranty diagnosis · Knowledge graph · GraphRAG · FMEA · Bayesian ranking
ETL · Neo4j · LangGraph · LLMOps · Redis · Runtime scale

How to use: Memorize §1 pitch → draw §3 diagrams → drill persona Q&A → practice scenarios.

Golden rule for every answer:

(1) plain English → **(2) why it matters** → **(3) how our app does it** → **(4) file/tool**

Plus WWWH: **What** · **Where** · **When** · **How** · **Why** (indexes worked example in §13 / docs/19).

1. Contents & interview map

Interviewer	Read first	Practice
Anyone (opening)	\$2 Elevator pitch + \$3 What we built	90-second story out loud
Academic / research	\$6 Theory + Academic Q&A;	Derive Bayes & RPN on whiteboard
Senior developer	\$5 Tools + \$7 Code spine + Dev Q&A;	Trace POST /diagnose in IDE
Senior architect	\$4 Architecture + Arch Q&A;	Draw ETL → Neo4j → API → UI
Industry / domain	\$3 Domain chain + Industry Q&A;	Asset → FM → part → claim

Sections: 2 Pitch · 3 Product domain · 4 Architecture · 5 Tools · 6 Theory · 7 Code · 8 Ontology/topology · 9 Persona Q&A; · 10 Scenarios · 11 Memory cards · 12 Glossary

2. The 90-second elevator pitch (memorize)

We built an **enterprise remote-diagnostics system** for warranty support. A customer or agent describes a problem (“washer won’t drain, E21”). We resolve the **product/asset**, walk a **Neo4j knowledge graph** of symptoms → failure modes → parts, score candidates with **FMEA + Bayes**, return ranked diagnosis, troubleshooting steps, and predicted parts, then optionally open a **claim** or **escalate**.

Knowledge is **not invented by an LLM at runtime** — it is **authored as blueprints**, **ETL’d** from PIM/FSM/Claims, **validated**, and **MERGED into Neo4j**. LLM (if enabled) only helps wording; **evidence is graph-native and deterministic**.

Non-technical one-liner

“We help warranty agents find the right fix and part by searching a product knowledge graph instead of guessing from a chatbot.”

Technical one-liner

“Deterministic GraphRAG over a warranty ontology in Neo4j, with FMEA/Bayesian ranking, enterprise ETL, and LLMOps-ready API.”

3. What the application does (domain)

3.1 The problem

When a product fails, support must answer: what unit is this, what is wrong, how do we confirm, which part, is it under warranty? Without a graph, people dig PDFs and tribal knowledge. We encode knowledge as a **graph** so answers are consistent, explainable, and fast.

3.2 Operational chain (whiteboard this)

```
Asset/Serial → Product/Model/SKU → Symptom + ErrorCode
→ FailureMode (diagnosis) → DiagnosticStep
→ Component (BOM) → Part → Claim / HistoricalResolution
```

Business question	Graph answer
What unit?	Asset → Product / BOUND_TO_SKU
What could be wrong?	Symptom -INDICATES→ FailureMode
How sure?	Edge confidence + FMEA + Bayes posterior
Which part?	REQUIRES_PART + BOM + claims precedent
Covered?	Warranty policy + asset status

Remember: Always narrate the chain Asset → Symptom → FailureMode → Part → Claim.

4. End-to-end architecture (as built)

4.1 Six layers

Layer	Responsibility	Key modules
L6 Experience	UI	Next.js frontend
L5 Orchestration	Agent, claims, warranty	diagnosis_service, LangGraph
L4 Intelligence	GraphRAG, FMEA/Bayes, trees, parts	graph_rag, reliability, ...
L3 Integration	Enterprise systems	connectors, CRM, mock :8090
L2 Knowledge store	Neo4j	populate_graph, neo4j_client
L1 Data platform	Blueprints, ETL	oem catalog, orchestrator

4.2 Complete system pipeline (diagram 39)

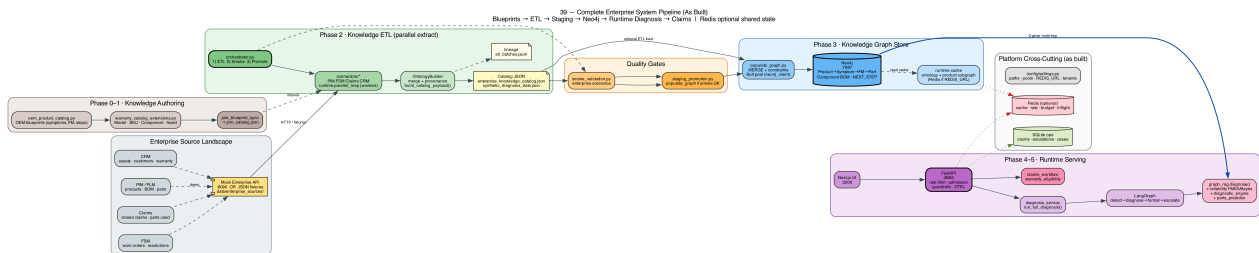


Fig. 1 — Sources → ETL → gates → Neo4j → API/UI → claims (+ Redis optional)

- **Author** OEM blueprints + warranty extensions (Model/SKU/Component)
- **Extract** PIM/FSM/Claims/CRM in parallel (parallel_map)
- **Transform** OntologyBuilder → catalog JSON + provenance
- **Gate** smoke validation → **Promote** MERGE into Neo4j
- **Serve** UI → API → LangGraph → GraphRAG Cypher multi-hop

4.3 Network topology (diagram 40)

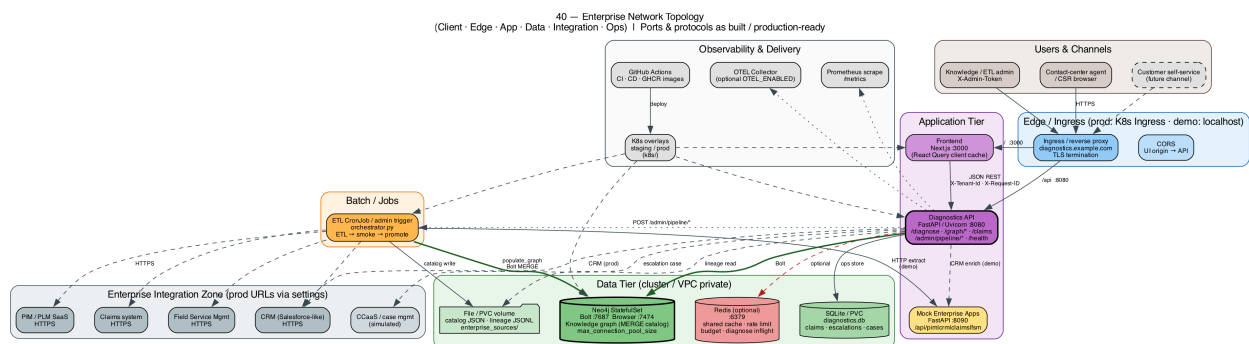


Fig. 2 — Enterprise network: clients, ingress, app, data, integration zone, ops

Component	Port	Role
Next.js UI	3000	Agent experience
FastAPI	8080	Diagnose, graph, claims, admin
Mock enterprise	8090	Simulated PIM/CRM/FSM/Claims
Neo4j	7687	Knowledge graph (Bolt)
Redis (optional)	6379	Shared cache / rate / budget

4.4 ETL → staging → fast traversal (diagram 41)

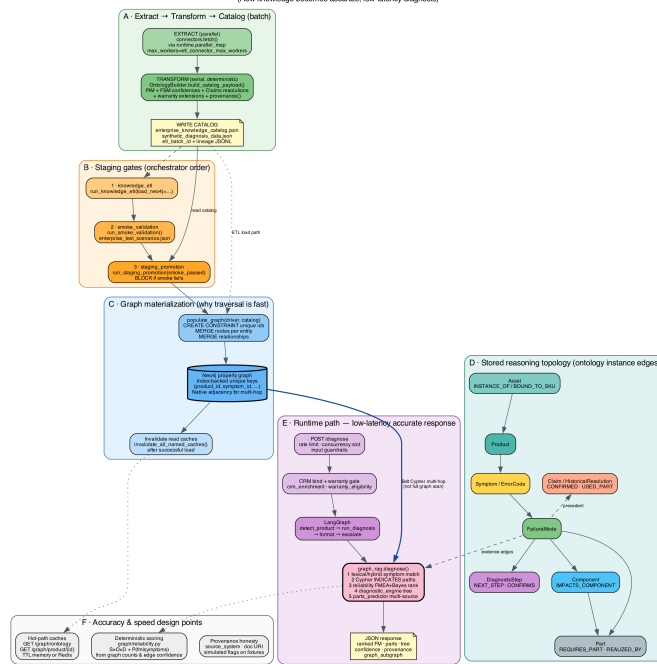


Fig. 3 — Why multi-hop diagnosis is fast: pre-built edges + unique keys + short paths

Remember: We don't scan the whole graph each request. We land on a product node, then walk short pre-built paths.

4.5 Ontology schema (ERD)

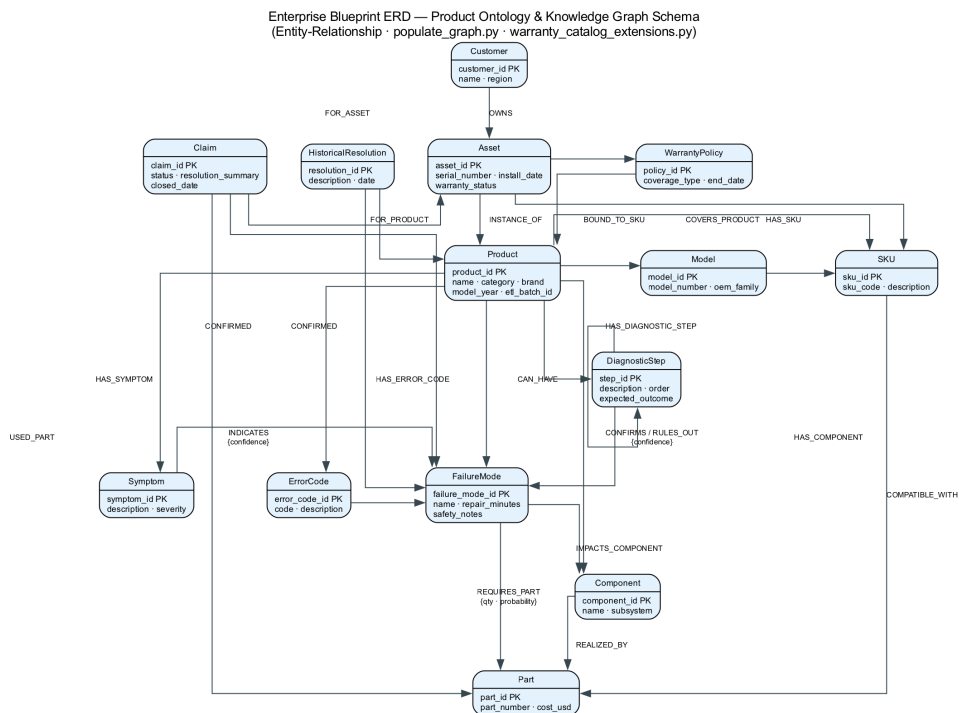


Fig. 4 — Entity-relationship contract of the warranty ontology

4.6 GraphRAG diagnosis flow

Figure 4: GraphRAG diagnose() — Internal Flow & Cypher Queries

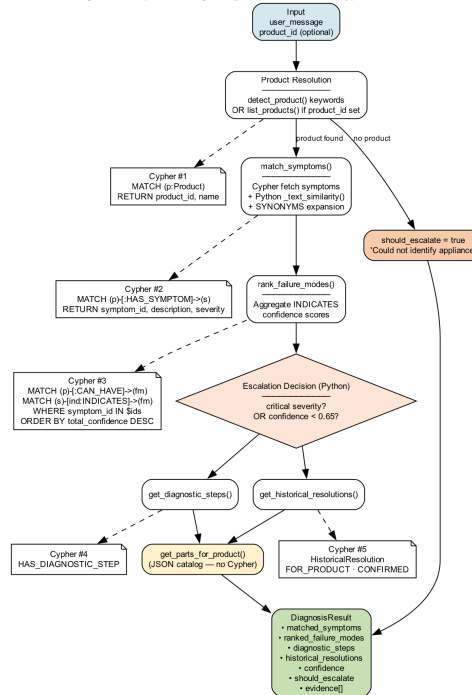


Fig. 5 — Runtime GraphRAG diagnosis path

5. Tools & technologies (plain English)

Tool	Plain English	Our role
Neo4j	Database for relationships	Stores nodes + edges for diagnosis
Cypher	SQL-for-graphs	Multi-hop evidence queries
FastAPI	Python web API	POST /diagnose, graph GETs
LangGraph	Workflow as steps	detect → diagnose → format → escalate
GraphRAG	Retrieve graph, ground answer	Cypher evidence + format
Redis	Shared fast memory (optional)	Multi-pod cache/rate/budget
FMEA	Reliability worksheet	S/O/D style signals
Bayes	Update beliefs with evidence	Rank failure modes
OWL/RDF	Formal ontology languages	Optional export, not runtime store
OTEL / Prom	Observability	Traces & metrics

Neo4j vs relational

Relational DBs excel at transactions and fixed joins. Multi-hop diagnostic paths (symptom→FM→component→part→claim) are natural in a property graph with typed edges and edge properties (confidence, probability). We still use SQLite for simple ops tables (claims/escalations); Neo4j is the diagnostic brain.

Remember: Is this an LLM chatbot? Core diagnosis is graph + math. LLM is optional and off by default.

6. Theory (academic + industry)

6.1 Ontology vs knowledge graph vs topology

Term	Meaning	Our app
Taxonomy	Category hierarchy	Product categories
Ontology	Types + allowed relations	Product, Symptom, FM, Part...

Term	Meaning	Our app
Knowledge graph	Ontology + instances	wm-001 + real edges in Neo4j
Topology (domain)	Structure/composition	BOM Component + diagnostic tree
Topology (infra)	Network of systems	Diagram 40 — not domain model

6.2 FMEA / FMECA

FMEA = Failure Mode and Effects Analysis. For each failure mode estimate **Severity**, **Occurrence**, **Detection**. Classic **RPN** = **S×O×D**. AIAG-VDA also uses Action Priority because raw RPN can rank-reverse. Our graph/reliability.py derives signals from graph data (symptom severity, claim counts, diagnostic coverage).

6.3 Bayesian diagnostic inference

$P(\text{failure mode} \mid \text{symptoms}) \propto P(\text{fm}) \times \prod P(s_i \mid \text{fm})$. Prior from history; likelihood from INDICATES.confidence; normalize across candidates. This is Pearl / AIMA-style belief update with evidence.

Remember: Whiteboard: two FMs, two symptoms, assign confidences, multiply × prior, normalize to 1.

6.4 ETL & systems concepts

- **ETL:** Extract (parallel I/O) → Transform (serial deterministic) → Load (MERGE)
- **Cache:** remember expensive stable reads (schema/subgraphs), not free-text diagnosis by default
- **Concurrency:** parallel independent fetches; keep one diagnosis ranking serial
- **Partition keys:** tenant|product for rate limits/caches; physical sharding later
- **Admission control:** cap concurrent diagnoses so Neo4j isn't overwhelmed

6.5 LLMOps snapshot

PromptOps (versioned prompts) · EvalOps (golden tests) · Guardrails (input/output) · FinOps (daily budget circuit breaker) · Gateway (model routing) · Observability (OTEL/metrics).

7. Code spine — how pieces connect

7.1 Batch path

```
orchestrator.run_all()
  knowledge_etl → parallel_map(fetch) → OntologyBuilder → catalog JSON
  smoke_validation (block promote if fail)
  staging_promotion → populate_graph() MERGE + constraints → Neo4j
  invalidate_all_named_caches()
```

Files: orchestrator.py, knowledge_etl.py, ontology_builder.py, populate_graph.py, oem_product_catalog.py

7.2 Online path (POST /diagnose)

```
api/main.diagnose
  → rate limit + concurrency slot + guard_request
  → CRM enrich + warranty gate
  → diagnosis_service.run_full_diagnosis
    → LangGraph: detect → tool_diagnose → format → escalate
      → graph_rag.diagnose (Cypher + reliability + tree + parts)
  → validate_output → DiagnoseResponse JSON
```

7.3 Mini Cypher

```
MATCH (p:Product {product_id:$pid})-[:HAS_SYMPTOM]->(s:Symptom)
MATCH (s)-[r:INDICATES]->(fm:FailureMode)
RETURN fm, r.confidence ORDER BY r.confidence DESC
```

8. Ontology, topology, RDF

Question	Short answer
Separate topology system?	No — BOM/Component is inside the ontology
W3C BOT?	Building spatial topology — not our domain
ISO 14224?	Equipment hierarchy + failure taxonomy — maps to us
OWL/RDF export?	Interchange/docs via rdf_ontology_export; runtime is Neo4j

9. Question bank by interviewer persona

9.1 Academic / theory

[ACADEMIC] Q: Formalize diagnosis as probabilistic inference.

A: Failure modes = hypotheses; symptoms = evidence. Naive Bayes: $\text{posterior} \propto \text{prior} \times \prod P(s_i | fm)$. Likelihoods from INDICATES.confidence; priors from claim/history frequency; normalize. Code: reliability.py + GraphRAG ranking.

[ACADEMIC] Q: Why is RPN criticized? What do you do?

A: Ordinal multiplication can rank-reverse. AIAG-VDA favors Action Priority. We expose RPN-like scores but rank diagnosis primarily by Bayesian posterior grounded in graph counts.

[ACADEMIC] Q: Ontology vs knowledge graph?

A: Ontology = TBox (schema of classes/properties). KG = TBox + ABox (instances). OWL export $\approx$ TBox; Neo4j holds operational ABox for all products.

[ACADEMIC] Q: Why multi-hop graph over pure vector RAG?

A: Typed edges encode expert causal/diagnostic constraints. Vectors help language match; final ranking stays on symbolic paths for explainability and audit.

[ACADEMIC] Q: How do you avoid LLM hallucination?

A: Facts from Neo4j only; LLM optional for phrasing; provenance trail; evals/guardrails; LLM off by default.

9.2 Senior developer

[SENIOR DEV] Q: Walk the code path of POST /diagnose.

A: api/main.diagnose → admission + guardrails → CRM/warranty → run_full_diagnosis → LangGraph → graph_rag.diagnose Cypher+rank → format → JSON. Name the files in order.

[SENIOR DEV] Q: How is ETL idempotent?

A: Neo4j MERGE on natural keys + uniqueness constraints. Re-promote updates properties, doesn't duplicate.

[SENIOR DEV] Q: Why parallel extract, serial transform?

A: Extract is I/O-bound and independent. Transform merges confidences and must stay deterministic.

[SENIOR DEV] Q: Redis without hard dependency?

A: Empty REDIS_URL uses in-process memory. Same interfaces for cache, rate limit, budget, concurrency. Fall back to memory if Redis errors.

[SENIOR DEV] Q: Breakpoint for wrong part prediction?

A: parts_predictor scoring; REQUIRES_PART/BOM edges; claim precedent; then check if wrong FM ranked upstream.

9.3 Senior architect

[ARCHITECT] Q: Trust boundaries?

A: Agents → Ingress → UI/API; Bolt private to Neo4j; outbound connectors; admin token for pipelines; Redis for multi-replica shared state. No public Bolt.

[ARCHITECT] Q: Why Neo4j over Postgres recursive CTEs?

A: Variable-length diagnostic/BOM paths, edge properties first-class, clearer domain model for multi-hop evidence. Postgres still fine for claim ledgers.

[ARCHITECT] Q: Safe knowledge promotion?

A: ETL → smoke scenarios → promote only if pass → MERGE with lineage batch id → invalidate caches.

[ARCHITECT] Q: Where does LLM fit?

A: Language edge tasks behind gateway with budget/evals — never sole source of failure-mode truth.

[ARCHITECT] Q: Multi-tenant evolution?

A: Logical partition keys now; next OIDC tenant claims + scoped queries; later Neo4j multi-DB if needed.

9.4 Industry / domain senior

[INDUSTRY] Q: How do you reduce wrong parts / MTTR?

A: Ranked failure modes + linked parts from graph and claim precedent; BOM path shows subsystem impact.

[INDUSTRY] Q: Alignment to FMEA / ISO 14224?

A: Failure modes + S/O/D-style signals; equipment hierarchy as Component/BOM; claims feedback as continuous improvement.

[INDUSTRY] Q: 10,000 SKUs — scale?

A: ETL by product line; cache hot subgraphs; index product_id; avoid global scans; async ETL workers when batch grows.

[INDUSTRY] Q: Salesforce / SAP integration?

A: Connector abstraction already exists. Swap mock HTTP for real APIs; OntologyBuilder is the anti-corruption layer.

10. Practice scenarios

A — CTO in 3 minutes

- Problem: inconsistent warranty diagnosis
- Approach: product knowledge graph + deterministic ranking
- Pipeline: enterprise systems → ETL → Neo4j → API
- Trust: provenance, gates, human escalation
- AI: LLM optional; evidence graph-first
- Scale: pools, caches, Redis, K8s shapes

B — Wrong diagnosis postmortem

- Wrong product resolution?
- Symptom match quality?
- INDICATES confidences / sparse priors?
- Part wrong but FM right? → parts_predictor

- Fix data quality vs algorithm

Closing master answer (if time is almost up)

“End-to-end: we author product diagnostic knowledge, ETL it through quality gates into Neo4j as a typed ontology of symptoms, failure modes, steps, and parts. At runtime, FastAPI and LangGraph call GraphRAG, which walks short Cypher paths and ranks failure modes with FMEA-style signals and Bayesian posteriors, then predicts parts and can open claims. Determinism, provenance, and optional LLM phrasing keep it enterprise-safe. Scale-out uses connection pools, caches, and Redis-backed limits for multi-replica APIs.”

11. Memory cards

Card	Memorize
Pipeline	Blueprints → Connectors → OntologyBuilder → Catalog → Smoke → Promote → Neo4j → GraphRAG
Ranking	$\text{Posterior} \propto \text{Prior} \times \prod P(\text{symptom} \text{FM})$
FMEA	$S \times O \times D \rightarrow \text{RPN (+ Action Priority)}$
Topology	BOM Component structure lives inside ontology — not a separate system
Ports	3000 UI · 8080 API · 8090 mock · 7687 Neo4j · 6379 Redis
Honesty	Demo fixtures may be simulated — labeled in provenance

Key files

Need	File
HTTP entry	api/main.py
Business rules	services/diagnosis_service.py
Agent flow	agents/diagnosis_graph.py
Brain	graph/graph_rag.py
Math	graph/reliability.py
Load graph	graph/populate_graph.py
ETL	enterprise_pipeline/.../knowledge_etl.py + orchestrator.py
Runtime scale	runtime/*

12. Glossary

Term	Meaning
Asset	Installed unit with serial number
SKU	Sellable revision of a model
Symptom	Observed problem
Error code	Device-reported code (e.g. E21)
Failure mode	Diagnosed fault type
Component	BOM subsystem
Part	Replaceable spare
Provenance	Where a fact came from
MERGE	Neo4j upsert by key
GraphRAG	Retrieve graph evidence to ground answers
LangGraph	Stateful multi-step agent workflow
RPN	Risk Priority Number $S \times O \times D$
Admission control	Limit concurrent heavy requests

13. Indexes & constraints (WWWH — interview ready)

Use five questions for any subsystem: What · Where · When · How · Why. Full write-up: docs/19-Indexes-Constraints-and-Lookup-Performance.md

	Neo4j uniqueness	SQLite ops
What	UNIQUE constraint on each *_id (backing unique index)	PK + status indexes
Where	populate_graph.create_constraints	utils/persistence.py
When	Every graph load/promote (not each diagnose)	First DB open
How	CREATE CONSTRAINT IF NOT EXISTS ... IS UNIQUE	CREATE INDEX idx_*_status
Why	Fast MATCH by id + no duplicates	Filter agent queues by status

Say in interview: We resolve product by unique product_id (constraint-backed index), expand HAS_SYMPTOM, then hybrid TF-IDF in Python. No full-text index today.

14. Red flags to avoid

Don't say	Say instead
"The LLM figures out the failure mode"	"Graph ranks FMs; LLM may phrase the answer"
"We built a separate topology product"	"Product structure is Component/BOM in the ontology"
"Redis is required"	"Redis is optional for multi-replica; memory works for demo"
"It's just RAG on PDFs"	"Structured multi-hop graph with typed evidence edges"
"Always 99% accurate"	"Deterministic given data; escalate when ambiguous"

Deeper reading in repo: docs/15 (ontology/RDF), docs/16 (runtime), docs/17 (landscape), PIPELINE-AND-MODULE-GUIDE.md · Graphviz diagrams 04, 34, 39–41.